NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

DIOGO HENRIQUE SILVESTRE MATOS CEBOLA

# PRIVACY-ENHANCED ONTOLOGIES

## A STUDY ON SECURE PROTOCOLS FOR EXPRESSIVE SPARQL QUERIES OVER ENCRYPTED ONTOLOGIES

MASTER IN COMPUTER SCIENCE

NOVA University Lisbon

***Draft:*** *February 15, 2022*

NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

DEPARTMENT OF
COMPUTER SCIENCE

# PRIVACY-ENHANCED ONTOLOGIES

## A STUDY ON SECURE PROTOCOLS FOR EXPRESSIVE SPARQL QUERIES OVER ENCRYPTED ONTOLOGIES

DIOGO HENRIQUE SILVESTRE MATOS CEBOLA

**Advisers:** Matthias Knorr
*Auxiliar Professor, NOVA University Lisbon*
João Leitão
*Auxiliar Professor, NOVA University Lisbon*
Henrique Domingos
*Associate Professor, NOVA University Lisbon*

MASTER IN COMPUTER SCIENCE

NOVA University Lisbon
***Draft:*** *February 15, 2022*

# Contents

# List of Figures

# List of Listings

# Acronyms

**DSSE** Dynamic Symmetric Searchable Encryption 3, 4

**FHE** Fully Homomorphic Encryption 3, 17

**GO** Gene Ontology 1

**HE** Homomorphic Encryption 2, 3, 6, 17
**HTML** HyperText Markup Language 9
**HTTP** Hypertext Transfer Protocol 17

**LFHE** Leveled-Fully Homomorphic Encryption 3, 17

**NCI** National Cancer Institute 1

**ORAM** Oblivious RAM 3
**OWL2** Web Ontology Language iv, v, 7, 8, 10, 11, 12, 13, 14, 15

**PEKS** Public Key Encryption with Keyword Searchable 3, 17
**PHE** Partially Homomorphic Encryption 3, 17

**RDF** Resource Description Language v, vi, 7, 8, 9, 10, 11, 15, 16, 18
**RDFS** Resource Description Framework Schema 7, 8, 9, 10, 11, 12

**SE** Searchable Encryption 2, 3, 6, 17
**SHE** Somewhat Homomorphic Encryption 3, 17
**SLA** Service Level Agreement 2
**SPARQL** SPARQL Protocol and RDF Query Language v, 3, 4, 7, 8, 15, 16, 17, 18
**SQL** Structured Query Language 15

**SSE** Symmetric Searchable Encryption 3, 4, 17
**STE** Structured Symmetric Encryption 3, 4
**SWRL** Semantic Web Rule Language 7

**TURTLE** Terse RDF Triple Language v, 8, 9, 12, 15

**URI** Uniform Resource Identifiers 6, 8, 12

**W3C** World Wide Web Consortium 6, 17
**WWW** World Wide Web 1

**XML** Extensible Markup Language 7, 9, 11

# 1

# Introduction

## 1.1 Context

In 2001, the term *Semantic Web* [4] was first used to define what would be the evolution of the World Wide Web (WWW). The traditional *Web* solved the challenge of distributing information at scale and globally, however it was designed for human use. Documents are easily accessible and readable by humans, but for automated agents little more than document retrieval is possible. Contrary to humans, machines cannot reason over the information they access, if they do not know the *semantics of the data*.

The *Semantic Web* addresses this issue, by redesigning the representation of information, in a way that not only preserves meaning and relationships but is also machine-readable. A concept is mapped to a unique identifier, and meaning is encoded in sets of triples, each triple consisting of a subject, predicate and object (similar to an elementary sentence). *Ontologies* provide a shared understanding of a domain by formalizing the specification of relevant concepts, and their relations, in such domain.

This novel method of knowledge representation differs from the previous, centralized way, and greatly simplifies the process of interconnecting different knowledge bases. Centralized representations tend to be more expensive to develop, usually needing to be built from scratch [25]. Furthermore, for sharing, both parties need to agree on the semantics. In contrast, using ontologies, information is semantically linked by default. In the *Semantic Web*, machines can truly communicate, share and reason about distinct knowledge bases.

Knowledge representation is essential in the industry [26] and ontologies help remove ambiguity in terminology. Nowadays, they are being developed and applied in a variety of domains [55] ranging from the health sector[1], to linguistics [2] and even being used by companies like NASA[3] [3].

[1] The NCI Thesaurus [34], developed by the National Cancer Institute (NCI) or the Gene Ontology (GO) [15], the world's largest source of information on the functions of genes, are some existing applications.

[2] The WordNet [21] is a lexical database for English where related words and concepts are semantically linked.

[3] Observing the visual representation provided by the Linked Open Data Cloud [19] is a good exercise to perceive the dimension of adoption of this new paradigm.

To developed and maintain ontologies, advanced semantic editors are the standard tool for professionals, with Protégé [24] being a very popular example. These provide numerous functionalities, usually based on plugins (e.g. automatic verification of errors and consistency in the design; debugging tools; query answering; graphic visualization of data).

With ontologies gradually growing in dimension, the industry is currently evolving towards the use of collaborative solutions [39, 38] and outsourcing heavy computations, relying on cloud infrastructures or other shared computing platforms. These approaches, albeit inevitable, present new security concerns and engineering challenges.

As of 2021, cloud adoption as risen more than what was previously forecasted, with 50 percent of all enterprise workloads hosted in *public clouds* and predictions of 7 percentage point gain in 2022 [33]. Research on secure cloud computing [8] studies such adoption but also discusses security concerns. Adoption can be explained with the cost efficiency[4], high scalability, flexibility and availability that cloud provides. Additionally, the pay-per-use model is an easy entry point to the ecosystem.

On the contrary, while delegation of responsibilities is beneficial for costs, it is prejudicial to control. Using cloud services implies trusting its owner. A Service Level Agreement (SLA) defines, in accordance to metrics, the service levels of the system that the vendor should provide (e.g. percentage of availability), while also defining penalties in case of not achieving what was agreed-on. However, the risk of loss of privacy and data breaches still exists. A problem that is especially concerning when sensitive data, such as health records, is being stored in such services. This is much more relevant in *public clouds* where, compared to *private clouds*, trust is distributed between various actors and not on a single organization [8].

In cloud storage services, administrators, or hacker with root rights, might have access to the data [6]. Additionally, these cloud services might outsource computations to third parties whom we do not trust. In this setting, we usually consider the *honest-but-curious*, or *passive*, adversary model. This adversary will follow the protocol properly but keep a record of all its intermediate computations [13]. Furthermore, it cannot do any other action except attempting to learn private information while observing views of the protocol execution [11].

A first approach to secure data in the cloud would be to use *Encryption at Rest*, which naively encrypts all data stored in the untrusted servers. If a user wants to do to write or search operation over the data, they will have to download all the data, and decrypt it. This incurs in high network requirements and is computationally expensive on the client-side, hindering all the benefits of using a cloud service to store the data. Nonetheless, this is the mechanism usually provided by database services [29, 28, 23].

Recently, active research is being done in the fields of Searchable Encryption (SE) and Homomorphic Encryption (HE).

[4]Cloud services reduce implementation and maintenance, power, cooling, floor space, storage and operational costs.

Homomorphic Encryption is a cryptographic method based on the concept of algebra homomorphism, where both decryption and encryption can be seen as an homomorphism of the relations between the plaintext and the ciphertext [1, 12]. Homomorphic Encryption, based on the type and number of operations allowed to be done on the encrypted data, can be classified as: Fully Homomorphic Encryption (FHE) supporting any number of operations with no restriction on the number of times; Somewhat Homomorphic Encryption (SHE) permitting some types of operations, a limited number of times; Partially Homomorphic Encryption (PHE) allowing one type of operation with no restriction on the number of times; and Leveled-Fully Homomorphic Encryption (LFHE) [7] supporting any number of operations, but with a pre-determined expected depth-$L$.

Searchable Encryption uses classical encryption schemes to perform encrypted search queries, sequentially over the ciphertext [35], or through the generation of searchable *encrypted indexes* accessible with *tokens*, generated via *trapdoor* functions. Searchable Encryption can be divided in Symmetric Searchable Encryption (SSE), when it is based on symmetric key cryptography, and Public Key Encryption with Keyword Searchable (PEKS), when asymmetric key algorithms are used. These techniques may leak some information about the *indexes*, *search pattern*, and *access pattern* during execution. Most approaches leak, at least, the *search* and *access patterns* [6].

Besides HE and SE, Oblivious RAM (ORAM) [14] could theoretically be used as an alternative. However, ORAM and HE are not efficient enough, when used exclusively, to be feasible yet [6] in real world applications. Also, SSE tends to be more practical than PEKS, due to the efficiency comparison between asymmetric and symmetric cryptographic schemes [30].

## 1.2 Problem

Ontologies usually only contain public, and non-sensitive data, but sometimes they are used alongside electronic health records and clinical decision systems. Furthermore, they can hold private value, due to the investment of time and money in their development.

Taking into considerations the efficiency and security trade-offs from the various alternative techniques, the SSE approach seems to be the most practical solution for protecting ontologies in untrusted servers. However, most SSE protocols only consider *unstructured* collections of documents with no semantic relationships. More recently, Chase and Kamara, on their work on Structured Symmetric Encryption (STE), generalized index-based SSE to support queries on *structured* data [9].

Ontologies can be represented as graphs and STE supports various types of query operations in graph-structured data. Nevertheless, STE with direct support for the query expressiveness required for searches over ontologies has not yet been studied, more specifically for languages like SPARQL [**Prudhommeaux2008-vf**, 37].

Furthermore, write operations that are supported in Dynamic Symmetric Searchable Encryption (DSSE) [16], have not yet been studied in the context of ontologies.

Additionally, information leakage can be a problem while trying to guaranty privacy of the topology of an ontology.

To conclude, we will focus on answering the following questions:

1. *How to support expressive queries and other operations in privacy-enhanced encrypted ontologies, using techniques such as Structured Symmetric Encryption and Dynamic Symmetric Searchable Encryption?*

2. *Besides preserving data privacy, how can we minimize information leakage and avoid possible exposure of the topology of the encrypted ontologies?*

## 1.3 Objectives

The main objective of this thesis will be to provide answers to the two questions defined in section 1.2 through a problem driven approach and guide future work in this field of research. We will first study a static STE approach focusing on support for SPARQL queries in a multi-user setting, later expanding to a dynamic setting. Finally, we will study how to minimize information leakage about the topology of the encrypted data.

## 1.4 Contributions

Our contributions will be the proposal of a protocol that extends upon STE and supports expressive and secure SPARQL queries over privacy-enhanced encrypted ontologies. We now present a summary of the contributions:

**SSE M/M Protocol for Privacy-Enhanced Ontologies** A novel multi-user protocol, based on STE, that supports SPARQL queries over encrypted ontologies, stored in untrusted servers.

**P1 - Static Setting** A protocol implementation for a static setting, supporting only search operations, based on STE.

**P2 - Dynamic Setting** A protocol implementation for a dynamic setting, supporting search, write and delete operations, based on STE and SSE.

All the prototypes developed will be available as open-source code in a public repository, alongside the benchmark results.

## 1.5 Document Organization

The remaining of the document will be organized as follows:

**Chapter 2** Analyses related work on *Semantic Web*, *Ontologies* and *Searchable Encryption*.

**Chapter 3** Presents requirements and informally defines the protocols and system models. Furthermore, in this chapter we describe our implementation and testing approaches.

**Chapter 4** Describes the work plan of the elaboration phase tasks.

# 2

# Related Work

On this chapter we will discuss the architecture and design of the *Semantic Web* as well as analyzing work already done in the fields of *Secure Computations*. The remaining of the chapter is organized as follows: on section 2.1 we discuss the *Semantic Web* stack, languages and tools; section 2.2 describes work on *Secure Computation*, with particular attention regarding HE and SE; finally, on section 2.3 we summarize and discuss which work is relevant.

## 2.1 Semantic Web: A layered architecture

In section 1.1, we have contextualized the main philosophy and design principles of the *Semantic Web*:

1. Usage of *labeled graphs* as the data model where concepts are represented as triples consisting of a subject, predicate and object (with nodes being objects and relations between such objects modelled as the edges);

2. All data items are identified by Uniform Resource Identifiers (URI);

3. Usage of ontologies to model semantics within a domain.

Additionally, the *Semantic Web* can also be seen as a stack of various layers [2] (figure 2.1). Most of these layers are standardized due to the collective work of the World Wide Web Consortium (W3C).

Briefly describing this layered architecture, at the root level, we have the URIs and a character set providing the baseline of communication, with distinguishable identification of resources and an agreed encoding.

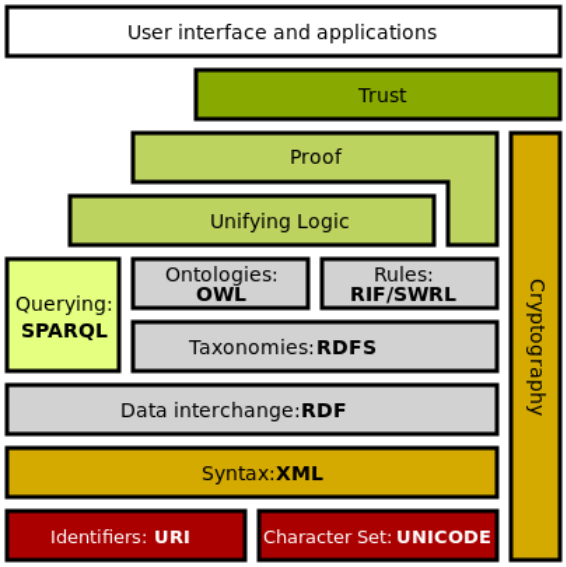


Figure 2.1: The *Semantic Web* stack [59].

At the syntax level we find the Extensible Markup Language (XML) [40]. XML supports the definition of unlimited *tags*, that can be placed in the text of a document. These *tags* facilitate the creation of hierarchy and give *structure* to the information.

From structured data we then define the basic data model using RDF [49]. With a XML like syntax, we can write statements about resources. Thus, embedding *semantics* within the information, as *metadata*.

The Resource Description Framework Schema (RDFS) [53] is a vocabulary description language to organize RDF resources into hierarchies, based on primitives such as *classes* and *properties*. It can be seen as a rudimentary *ontology language* [2].

Ontologies are defined with the OWL2 [42]. Like RDFS, it is also a vocabulary description language, but provides more expressiveness (e.g. relations between classes, *cardinality*, *equality*, richer properties with characteristics).

RDF triples can be interpreted as logical binary predicates, making reasoning possible over RDF-based data. This allows us to infer and validate ontologies, and also provide explanations on operations' results. The rules layer is a reference to this specific part of the architecture stack. Many rule languages, like the Semantic Web Rule Language (SWRL) [58], facilitate interaction based on *descriptive-logic*.

To query and retrieve any data based on RDF we use SPARQL.

The Proof and Unifying Logic layers include the notion of interconnectivity of terminologies between *knowledge bases*, with explainable proofs to operations.

Only with secure cryptographic schemes can we reach the trust layer. The security layer is still not standardized.

The top layer, references all the applications and interfaces that use this new paradigm of data representation.

In the following of this section we will describe, in more detail, some languages and

tools. We will focus on RDF in section 2.1.1; on RDFS, in section 2.1.2; on OWL2, in section 2.1.3; on SPARQL, in section 2.1.4; on common tools and software, in section 2.1.5.

### 2.1.1 RDF: Resource Description Framework

#### 2.1.1.1 Overview

RDF [49] is a flexible *domain independent* data model. The RDF basic primitives are *resources*, *Properties* and *literals*. We can construct *statements* about resources by organizing these basic primitives in sets of *triples*. A triple is composed by a *subject*, a *predicate* and an *object*. In a statement (triple) about a subject (always a resource) we define the value (object) of a property (predicate), which can be another resource or a literal.

Literals are atomic values (e.g. strings, numbers). Resources and properties, respectively, model the "things" and relationships between such "things". Both are identified using URIs. Resources without a URI are called *blank nodes* (or *bnodes*).

To make a statement $B$ about a statement $A$ we can use a technique called *reification*. Reification consists in: firstly, defining a statement $B$ where the object $objA$ represents the statement $A$; secondly, creating the properties *subject*, *predicate* and *object* mapped from $objA$ to the original subject, predicate (now a resource), and original object of statement $A$ [2].

#### 2.1.1.2 Syntax

**Graphical** RDF being a data model, can be written using different syntaxes. Graphically we can represent statements as a labeled and directed graph. The *nodes* are labelled with the resources URIs, a literal value or left blank, whereas *arcs* are labelled with the properties URIs. The arcs are directed from subject to object of a statement.

1) *TODO:* ADD graphical example

T[1]

**Text-Based** There are multiple text-based syntaxes for RDF. Some of the most commonly used are TURTLE [51] (listing 2.1), and its extension for named graphs *TriG* [50]. In TURTLE, URIs are enclosed by angle brackets. The subject, property, and object are written in this order. Statements are finished by a period. Literals are surrounded in quotes with the type of data specified with a URI. The data value and type are separated by ^^ (if no data type is mention, the value is inferred as being a string). We can also define URI abbreviations (using @prefix, followed by the prefix to substitute the URI, a colon, the URI and a period) and simplify statements, for easier readability (e.g.using a semicolon to make more than one statement about the same subject, defining statements with nested blank nodes using square brackets). Comments start with #.

```
# FCT is located in Almada
<http://www.my-custom-domain/faculties.ttl#FCT> <http://www.my-custom-domain/ontology/
    ↪ location> <http://www.my-custom-domain/resource/Almada> .

# FCT has 8.000 students
<http://www.my-custom-domain/faculties.ttl#FCT> <http://www.my-custom-domain/faculties.
    ↪ ttl#asNumberOfStudents> "8000"^^<http://www.w3.org/2001/XMLSchema#integer> .
```

Listing 2.1: Simple RDF example, written in TURTLE syntax.

*XML/RDF* [52] is a standardized XML-based syntax and *RDFa* [54] permits RDF to be embedded in the HyperText Markup Language (HTML)[1].

### 2.1.2 RDFS: RDF Schema

#### 2.1.2.1 Overview

RDF is a flexible data-model because it is not domain-specified. However, for interconnectivity of different RDF knowledge bases, semantics for restricting meaning in a specific domain are needed.

RDFS [53] does that by providing vocabulary to describe groups of related resources and the relationships between these resources. This vocabulary is a set of RDF resources identified under the `http://www.w3.org/2000/01/rdf-schema#`.

#### 2.1.2.2 RDFS primitives

*Classes* group resources together, the *instances* of the class. *Properties* are defined by specifying to which classes of resource they may apply, using the mechanism of *domain* and *range*. This allows for easy extension, by not having to redefine classes wherever a new property, that applies to an existing class, is needed.

To state that a resource is an instance of a class we use the *rdf:type* property.

The *rdfs:subClassOf* is used to define that all the instances of a class are instances its *superclasses*: if *A* rdfs:subClassOf *B*, then instances of *A* are also instances of *B*. The *rdfs:subPropertyOf* states that instances related by a property are also related by its *superproperties*: if *A* rdfs:subPropertyOf *B*, then instances related to *A* are also related to *B*.The properties rdfs:subClassOf and rdfs:subPropertyOf are *transitive*, and there are no restrictions on the number of superclasses or superproperties that, respectively, a class or a property can have.

The *rdfs:domain* property defines that any resource with a given property is an instance of the domain classes that the property applies to. The *rdfs:range* property specifies that the values of the property are instances its range classes.

In tables 2.1 and 2.2, we summarize the core RDFS classes and properties. In figure ..., we extend the RDF graphical example ... with RDFS.

[1] We recommend the curious reader to read the W3C recommendations to better learn the syntaxes.

| RDFS Core Classes | |
|---|---|
| **Class** | **Description** |
| rdfs:Resource | The class resource, everything. |
| rdfs:Class | The class of classes. |
| rdfs:Literal | The class of literal values, e.g. textual strings and integers. |
| rdf:Property | The class of RDF properties. |
| rdfs:Statement | The class of RDF statements. |

Table 2.1: RDFS Core Classes and respective description (rdfs:comment).

| Core RDFS Properties | | | |
|---|---|---|---|
| **Property** | **Description** | **Domain** | **Range** |
| **Relationship Definition** | | | |
| rdf:type | The subject is an instance of a class. | rdfs:Resource | rdfs:Class |
| rdfs:subClassOf | The subject is a subclass of a class. | rdfs:Class | rdfs:Class |
| rdfs:subPropertyOf | The subject is a subproperty of a property. | rdf:Property | rdf:Property |
| **Property Restriction** | | | |
| rdfs:domain | A domain of the subject property. | rdf:Property | rdfs:Class |
| rdfs:range | A range of the subject property. | rdf:Property | rdfs:Class |
| **Reification** | | | |
| rdf:subject | The subject of the subject RDF statement. | rdf:Statement | rdfs:Resource |
| rdf:predicate | The predicate of RDF statements. | rdf:Statement | rdfs:Resource |
| rdf:object | The object of RDF statements. | rdf:Statement | rdfs:Resource |
| **Utility** | | | |
| rdfs:seeAlso | Further information about the subject resource. | rdfs:Resource | rdfs:Resource |
| rdfs:isDefinedBy | The definition of the subject resource. | rdf:Resource | rdfs:Resource |
| rdfs:comment | A description of the subject resource. | rdf:Resource | rdfs:Literal |
| rdfs:label | A human-readable name for the subject. | rdf:Resource | rdfs:Literal |

Table 2.2: RDFS Core Properties and respective description (rdfs:comment).

### 2.1.3 OWL2: Web Ontology Language

#### 2.1.3.1 Overview

An *ontology* was defined by Studer et al. as "a formal, explicit specification of a shared conceptualisation" [36]. An ontology should specify consensual and machine-readable knowledge by means of a formal definition of concepts, where concepts have explicit types and restrictions. Therefore, an *ontology language* should provide a well-defined syntax, formal semantics, convenient and sufficient expressiveness and efficient reasoning support.

RDFS can be considered a very limited *ontology language*. OWL2 [42] provides a number of additional features (e.g. cardinality constraints, class equivalence, intersection and disjunction). OWL2 does not simply extend RDFS, because RDFS primitives are too expressive. This would make reasoning very inefficient and computationally expensive [2]. As such, OWL2 is divided in two sublanguages: *OWL2 Full* [46], a direct extension of RDFS with all OWL2 semantics; *OWL2 DL* [41], a language with some restrictions on the way OWL2, RDF and RDFS primitives may be used.

The correspondence theorem of [41] states that: given an *OWL2 DL* ontology, inferences drawn using its *Direct Semantics* will still be valid if the ontology is mapped into an RDF graph and interpreted using the *RDF-Based Semantics*, the semantics of *OWL2 Full*.

OWL2 can be expressed with any valid RDF syntax, however there are various specific syntaxes: the Functional-Style Syntax [47], which is very compact and was the language used in the formal specification of the language; the Manchester Syntax [43], designed to be human-readable (it is very popular in user interfaces of semantic editors); and the OWL/XML Syntax [48], which allows easier processing with XML tools.

Figure 2.2 details the structure of OWL2. Finally, *OWL2* can have different specification, called profiles [44], their coverage is not on the scope of this work.

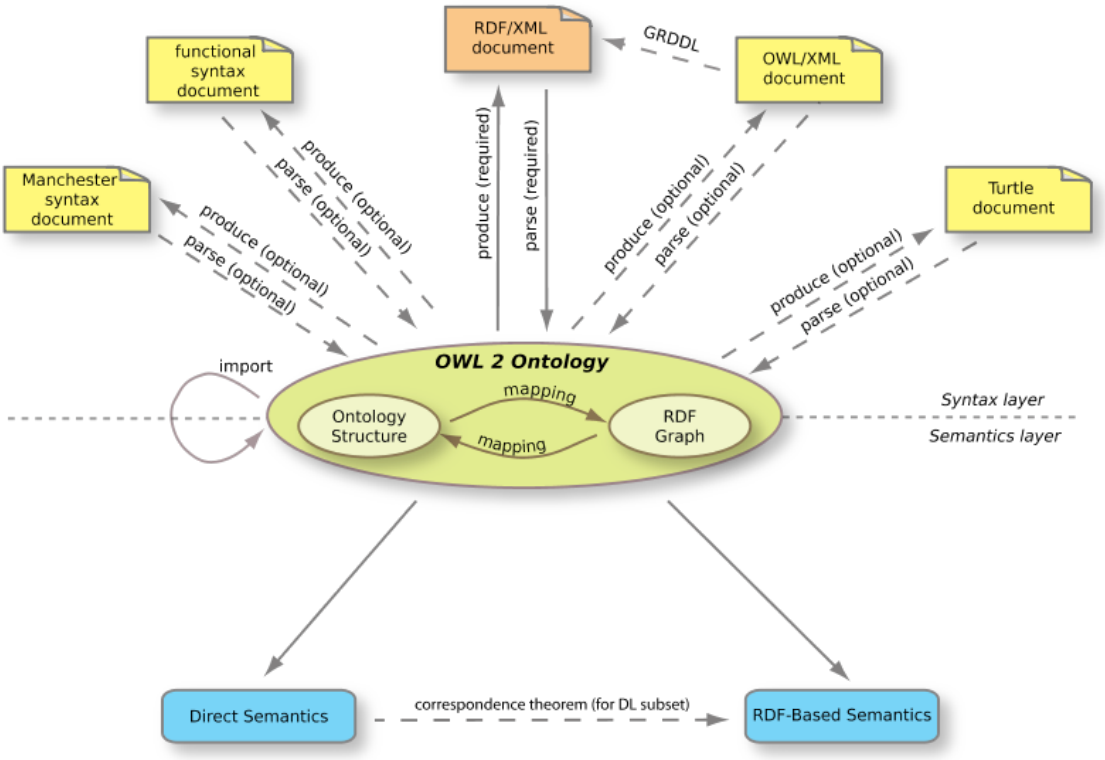


Figure 2.2: The OWL2 Structure [42].

#### 2.1.3.2 Owl Language

In OWL2 the term individual is equivalent to instance in RDFS. An *assertion* is a statement about a resource. *Expressions* are a combination of classes, properties and instances. An *axiom* relates such expressions to classes [2].

[2] For an overview of the language we recommend the reader to consult the reference guide [45].

```
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

# The base namespace of the FCT ontology
@base<http://www.my-custom-domain/fct.ttl> .

<http://www.my-custom-domain/fct.ttl>
    rdf:typeowl:Ontology ;
    rdfs:label"FCT Ontology"^^xsd:string ;
    rdfs:comment "An example OWL2 ontology"^^xsd:string ;
    owl:versionIRI <http://www.my-custom-domain/fct.ttl#1.0> .

# Class Expression
_:x rdf:type owl:Class ;
    owl:unionOf ( :Mandatory :Optional ) .

# Class Axiom
:Course owl:equivalentClass _:x .

# Class Assertion
:class101 rdf:type :Course .
```

Listing 2.2: Simple OWL2 example, written in TURTLE syntax.

#### Individual Assertions

In OWL2 *assertions* about individual resources can be used to define their type, class membership, properties and even provide ways to define identity and negative statements. More specifically on the identity statements, because OWL2 has an open world assumption, we can never differentiate two individuals just because they have different URIs. To do so we use owl:differentFrom. To explicitly state negative facts we use owl:NegativePropertyAssertion [2].

#### Classes

In OWL2 every resource that is of type owl:Class is a class. The two classes defined by default are the *owl:Thing* and *owl:Nothing*. Respectively, these two specify the superclass of all classes and the subclass of all classes: every individual is a member of owl:Thing and no individual can be member of owl:Nothing. This is very useful for reasoning, because inconsistency is detected when classes are equivalent to owl:Nothing (have no individuals in their membership).

**Class Axioms** In OWL2 we can relate *expressions* to classes using *axioms*. These can define:

- *Subclass Relations*, with equivalent usage like in RDFS;
- *Class Equivalence*, in equivalent classes every individual must be a member of each class;

- *Enumerations*, a type of class definition via explicitly stating all individuals who belong to the class membership;
- *Intersection*, in a class described as the intersection of various classes, all of its members must be also individuals belonging to the membership of each of those classes;
- *Union and Disjoint Union*, in a class described as the union of various classes, every individual belonging to its membership must also belong to the membership of at least one of those classes. In the particular case of disjoint union, the intersection of the classes is equivalent to owl:Nothing;
- *Disjoint Classes*, in disjoint classes no individual can be a member of both class (the intersection of the classes is equivalent to owl:Nothing);
- *Complement*, complement classes contain all the individual such that the union of both classes is equivalent to owl:Thing.

**Properties**

OWL2 has three types of properties: *object* properties, which relate individuals to other individuals; *datatype* properties, which relate individuals to literals; and *annotation* properties; which are used to specify human-readable labels, comments and explanations.

The *top* and *bottom* properties specify, respectively, properties that relate to all and no individuals: owl:topObjectProperty is the superproperty of all owl:ObjectProperties and owl:bottomObjectProperty relates to no individual; owl:topDataProperty is the superproperty of all owl:DatatypeProperties, relating an individual to any possible literal value, and owl:bottomDataProperty relates no individual to any literal value.

Properties can have different characteristics:

- *Transitive Properties*, if $a$ relates to $b$ via property $p$ and $c$ relates to $a$ via property $p$, then $c$ relates to $b$ via property $p$;
- *Symmetric Properties*, if $a$ relates to $b$ via property $p$ then inverse is true;
- *Asymmetric Properties*, if $a$ relates to $b$ via property $p$ then the inverse is $\perp$;
- *Functional Properties*, if for each instance the property can only have one unique value;
- *Inverse-functional Properties* if the object of a property statement relates to a single subject;
- *Reflexive Properties*, if every individual relates to itself via the property;
- *Irreflexive Properties*, if no individual relates to itself via the property.

**Property Axioms** Just like class axioms, we can specify *property axioms* to describe the way properties relate to classes and other properties. These allow us to define:

- *Domain and Range*, used to determine class membership of individuals;
- *Inverse Relationship*, used to specify the inverse of the property;
- *Equivalence*, used to specify equivalent properties. Individuals related by one property will always relate by all equivalent properties;
- *Disjointness*, used to specify disjoint properties, where individuals related by one property can never be related by the other;
- *Property Chains*, used to aggregate connected properties under another property.

**More Granular Class Axioms**

We can write more granular class definitions in OWL2 than by simply defining class axioms. This is achieved by combining class and property axioms. It is done by defining a class axiom that relates to an anonymous class (owl:Restriction) which contains axioms on its properties. The anonymous class definition captures all individuals that satisfy such restrictions. As such we can define:

- *Universal and Existential Restrictions* which the members of the class must satisfy. The class is defined as a subclass of an anonymous class with property restrictions on, respectively, owl:allValuesFrom and owl:someValuesFrom;
- *Necessary and Sufficient Conditions* similar to the above restriction but using a class equivalence axiom instead of rdfs:subClassOf;
- *Value, Cardinality, Data Range and Datatype Restrictions* on the anonymous class, limiting the value, cardinality of the value, range of possible values and the datatype of the property value relating to members of the superclass;
- *Self Restrictions* on the individuals captured by the anonymous class.

#### 2.1.3.3 OWL2 DL

Contrary to *OWL2 Full*, *OWL2 DL*, being less expressive, retains *decidability*. In this section, we specify the restrictions imposed by *OWL2 DL* [2]:

- The application of OWL2 primitives to each other is not allowed; only classes and non-literals resources may be defined, with all classes being individuals of owl:Class;
- Properties are disjoint individuals of either owl:ObjectProperty or owl:DatatypeProperty;
- Annotation properties are ignored by reasoners;

- Properties, for which range includes non-literal resources, are separated from those which relate to literal values;
- A resource can only be a class, property, or instance at the same time simultaneously, classes, properties or instances with the same name will be treated as distinct (a mechanism called punning);
- Only functional properties can be assigned to datatype properties;
- Only object properties can have an inverse;
- Property chains can only involve object properties;
- Self restrictions on datatype properties are not allowed.

### 2.1.4 SPARQL

#### 2.1.4.1 Overview

In the previous section we've discussed the structure of OWL2. OWL2 documents have a direct mapping to RDF documents. As such, we can publish and query OWL2 ontologies using the standard approach for RDF data.

RDF triples are stored in *triple stores*, specialized databases for this type of data. The underlying architecture of such databases varies, from relational to NoSQL approaches [10, 22]. The recommended query language to use is SPARQL [37]. It provides a range of operations over published RDF graphs, accessible via SPARQL endpoints: from read queries [57] to write operations [56, 27].

#### 2.1.4.2 SPARQL Queries

**Mathematical Basis** An isomorphism of two graphs is an adjacency preserving bijection between their vertex sets [20].

The basis of SPARQL queries is finding matches, on triple patterns, also referred as *Basic Graph Pattern*, in a RDF graph [17, 10]. This problem is equivalent to the subgraph isomorphism problem, a generalization of the graph isomorphism problem: given two graphs $G1$ and $G2$, we try to find if $G1$ contains a subgraph that is isomorphic to $G2$. In the context of SPARQL queries, $G1$ is the published RDF graph and $G2$ is the query triple pattern.

**Syntax** In a SPARQL query, we define variables, which will map onto the values of the corresponding matching triples. We call the result set of each variable its *bindings*. *SPARQL* queries have a similar structure to the Structured Query Language (SQL) queries: we define the operation to execute using a keyword, followed by the projection of variables to be shown in the result, the keyword *WHERE*, the triple pattern to match, enclosed in brackets, and, optionally, some modifiers of the result. The SPARQL syntax is very similar to TURTLE, so we can also specify prefixes for namespaces.

The basic *SPARQL 1.1 Query Language* [57] provides the following query operations:

- *SELECT*, used to extract values from a RDF graph. The result is a table with the variable bindings;
- *ASK*, which returns a boolean, true if there was a match in the RDF graph, false otherwise;
- *CONSTRUCT*, which returns a set of triples from the constructions of a new RDF graph, based on the matching pattern;
- *DESCRIBE*, which usually return a set of RDF triples that describes the matching patterns.The results depend on implementation [32].

In listing 2.3 we show a simple query. However, SPARQL provides additional features for more complex queries (listings 2.4 and 2.5): definition of the disjunction and conjunction of triple patterns, using the keywords *UNION* and *AND*; definition of optional patterns, using the *OPTIONAL* construct; filtering bindings, using the *FILTER* construct; elimination of duplicates with *DISTINCT*; application of aggregate and various functions to variables; matching regex patterns; ordering of results with *ORDER BY*; and, usage of named graphs, with the *FROM* and *FROM NAMED* constructs.

```
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix fct: <http://www.my-custom-domain/fct.ttl>

# Operation and variables
SELECT ?opt_course ?man_course ?popular_course
# Query Pattern
WHERE
{
    ?opt_course rdf:type fct:Optional .
    ?man_course rdf:type fct:Mandatory .
}
```

Listing 2.3: Simple SPARQL example

```
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix fct: <http://www.my-custom-domain/fct.ttl>

# Operation and variables
SELECT DISTINCT ?popular_course
# Query Pattern
WHERE
{
    ?popular_course fct:hasStudentEnrolled ?num_students
    FILTER (?num_students > 60).
}
# Optional Modifiers
LIMIT 20
```

Listing 2.4: SPARQL query with filters and modifiers example

```
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix fct: <http://www.my-custom-domain/fct.ttl>
@prefix nova: <http://www.my-custom-domain/nova.ttl>

# Operation and variables
SELECT DISTINCT ?optional_course
# Query Pattern
WHERE
{
    {?optional_course rdf:type fct:Optional}
    UNION
    {?optional_course rdf:type nova:Elective}
}
```

Listing 2.5: SPARQL a more complex pattern example

Besides read operations, *SPARQL Update*, provides additional options for inserting, publishing and deleting data in triple stores. For operations over the Hypertext Transfer Protocol (HTTP), *SPARQL 1.1 graph store HTTP protocol* has similar, but more limited, features[3].

### 2.1.5 Tools & Development Software for Ontologies

## 2.2 Secure Computations

### 2.2.1 Homomorphic Encryption

Homomorphic Encryption is a cryptographic method based on the concept of algebra homomorphism, where both decryption and encryption are seen as homomorphisms of the relations between the plaintext and the ciphertext [1, 12]. HE allows for computations done over encrypted data to be preserved after decryption of the ciphertext. An encryption scheme is homomorphic if such operations preserve additive or multiplicative properties. HE is classified in four main types: FHE supporting any number of operations with no restriction on the number of times; SHE permitting some types of operations, a limited number of times; PHE allowing one type of operation with no restriction on the number of times; and LFHE [7] supporting any number of operations, but with a pre-determined expected depth-$L$.

### 2.2.2 Searchable Encryption

Searchable Encryption uses classical encryption schemes to perform encrypted search queries. Searchable Encryption can be divided in SSE, when it is based on symmetric key cryptography, and PEKS, when public key algorithms are used. These techniques may leak some information about the *indexes*, *search pattern*, and *access pattern* during execution.

[3] Full examples and syntax can be found in the W3C recommendations.

### 2.2.3 Symmetric Searchable Encryption

#### 2.2.3.1 Structured Searchable Encryption

#### 2.2.3.2 Dynamic Symmetric Searchable Encryption

## 2.3 Summary

SPARQL has been widely studied in the literature. There has research on its semantics and complexity [31], on efficient processing of its queries [18] and survey on its patterns of use [5, 32]. As far as we know, there have been no works about the usage of SPARQL with encrypted RDF data.

# 3

# Approach to Elaboration

## 3.1 System Requirements

## 3.2 Privacy Guaranties

## 3.3 Architecture & System Model

### 3.3.1 Implementation Guidelines

### 3.3.2 Validation

# 4

# Work Plan

# Bibliography


[1] A. Acar et al. "A survey on homomorphic encryption schemes: Theory and implementation". In: vol. 51. 2018. DOI: 10.1145/3214303 (cit. on pp. 3, 17).

[2] G. Antoniou et al. "A Semantic Web Primer (Third Edition)". In: *The MIT Press* (2012), p. 287 (cit. on pp. 6–8, 10, 12, 14).

[3] N. Ashish. "Semantic-Web Technology: Applications at NASA". In: *Dagstuhl Seminar Proceedings*. 2005 (cit. on p. 1).

[4] T. Berners-Lee, J. Hendler, and O. Lassila. *The semantic web*. Vol. 284. 2001. DOI: 10.1038/scientificamerican0501-34 (cit. on p. 1).

[5] A. Bonifati, W. Martens, and T. Timm. "An Analytical Study of Large SPARQL Query Logs". In: (Aug. 2017). arXiv: 1708.00363 [cs.DB] (cit. on p. 18).

[6] C. Bösch et al. "A Survey of Provably Secure Searchable Encryption". In: *ACM Comput. Surv.* 47.2 (Aug. 2014), pp. 1–51. ISSN: 0360-0300. DOI: 10.1145/2636328 (cit. on pp. 2, 3).

[7] Z. Brakerski, C. Gentry, and V. Vaikuntanathan. "(Leveled) Fully Homomorphic Encryption without Bootstrapping". In: *ACM Trans. Comput. Theory* 6.3 (July 2014), pp. 1–36. ISSN: 1942-3454. DOI: 10.1145/2633600 (cit. on pp. 3, 17).

[8] M. Carroll, A. Van Der Merwe, and P. Kotzé. "Secure cloud computing: Benefits, risks and controls". In: *2011 Information Security for South Africa - Proceedings of the ISSA 2011 Conference*. 2011. DOI: 10.1109/ISSA.2011.6027519 (cit. on p. 2).

[9] M. Chase and S. Kamara. "Structured encryption and controlled disclosure". In: *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*. Vol. 6477 LNCS. 2010. DOI: 10.1007/978-3-642-17373-8_33 (cit. on p. 3).

[10] O. Curé and G. Blin. *RDF Database Systems: Triples Storage and SPARQL Query Processing*. en. Morgan Kaufmann, Nov. 2014. ISBN: 9780128004708 (cit. on p. 15).

[11] D. Evans, V. Kolesnikov, and M. Rosulek. "A Pragmatic Introduction to Secure Multi-Party Computation". In: *Foundations and Trends® in Privacy and Security* 2.2-3 (2018), pp. 70–246. ISSN: 2474-1558. DOI: 10.1561/3300000019 (cit. on p. 2).

[12] C. Fontaine and F. Galand. "A survey of homomorphic encryption for nonspecialists". In: *Eurasip Journal on Information Security* 2007 (2007). ISSN: 2510-523X. DOI: 10.1155/2007/13801 (cit. on pp. 3, 17).

[13] O. Goldreich. *Foundations of Cryptography: Volume 2, Basic Applications*. en. Cambridge University Press, 2001. ISBN: 9780521830843 (cit. on p. 2).

[14] O. Goldreich and R. Ostrovsky. "Software protection and simulation on oblivious RAMs". In: *J. ACM* 43.3 (May 1996), pp. 431–473. ISSN: 0004-5411. DOI: 10.1145/233551.233553 (cit. on p. 3).

[15] M. A. Harris et al. "The Gene Ontology (GO) database and informatics resource". en. In: *Nucleic Acids Res.* 32.Database issue (Jan. 2004), pp. D258–61. ISSN: 0305-1048, 1362-4962. DOI: 10.1093/nar/gkh036 (cit. on p. 1).

[16] S. Kamara, C. Papamanthou, and T. Roeder. "Dynamic searchable symmetric encryption". In: *Proceedings of the 2012 ACM conference on Computer and communications security*. CCS '12. Raleigh, North Carolina, USA: Association for Computing Machinery, Oct. 2012, pp. 965–976. ISBN: 9781450316514. DOI: 10.1145/2382196.2382298 (cit. on p. 3).

[17] G. Ladwig. *Efficient Optimization and Processing of Queries Over Text-rich Graph-structured Data*. en. KIT Scientific Publishing, 2013. ISBN: 9783731500155. DOI: 10.5445/KSP/1000034423 (cit. on p. 15).

[18] A. Letelier et al. "Static analysis and optimization of semantic web queries". In: *ACM Trans. Database Syst.* 38.4 (Dec. 2013), pp. 1–45. ISSN: 0362-5915. DOI: 10.1145/2500130 (cit. on p. 18).

[19] J. P. McCrae. *The linked open data cloud*. en. https://lod-cloud.net/. Accessed: 2022-2-8 (cit. on p. 1).

[20] B. D. McKay and A. Piperno. "Practical graph isomorphism, II". In: *J. Symbolic Comput.* 60 (Jan. 2014), pp. 94–112. ISSN: 0747-7171. DOI: 10.1016/j.jsc.2013.09.003 (cit. on p. 15).

[21] G. A. Miller. "WordNet: a lexical database for English". In: *Commun. ACM* 38.11 (Nov. 1995), pp. 39–41. ISSN: 0001-0782. DOI: 10.1145/219717.219748 (cit. on p. 1).

[22] G. E. Modoni, M. Sacco, and W. Terkaj. "A survey of RDF store solutions". In: *2014 International Conference on Engineering, Technology and Innovation: Engineering Responsible Innovation in Products and Services, ICE 2014*. 2014. DOI: 10.1109/ICE.2014.6871541 (cit. on p. 15).

[23] MongoDB, Inc. *MongoDB Manual: Encryption at rest*. en. https://docs.mongodb.com/manual/core/security-encryption-at-rest/. Accessed: 2022-2-9 (cit. on p. 2).

[24] M. A. Musen and Protégé Team. "The Protégé Project: A Look Back and a Look Forward". en. In: *AI Matters* 1.4 (June 2015), pp. 4–12. ISSN: 2372-3483. DOI: 10.1145/2757001.2757003 (cit. on p. 2).

[25] R. Neches et al. "Enabling technology for knowledge sharing". In: *AI Magazine* 12.3 (1991). ISSN: 0738-4602 (cit. on p. 1).

[26] N. Noy et al. "Industry-scale knowledge graphs: Lessons and challenges". In: *Commun. ACM* 62.8 (2019). ISSN: 0001-0782, 1557-7317. DOI: 10.1145/3331166 (cit. on p. 1).

[27] C. Ogbuji. *SPARQL 1.1 graph store HTTP protocol*. en. https://www.w3.org/TR/sparql11-http-rdf-update/. Accessed: 2022-2-4. Mar. 2013 (cit. on p. 15).

[28] Oracle Corporation and/or its affiliates. *MySQL :: MySQL 8.0 reference manual :: 15.13 InnoDB data-at-rest encryption*. en. https://dev.mysql.com/doc/refman/8.0/en/innodb-data-encryption.html. Accessed: 2022-2-9 (cit. on p. 2).

[29] Oracle Corporation and/or its affiliates. *MySQL enterprise encryption*. en. https://www.mysql.com/products/enterprise/encryption.html. Accessed: 2022-2-9 (cit. on p. 2).

[30] W. Pearson. *Cryptography and Network Security: Principles and Practice, Global Edition*. 7th ed. Pearson Education, 2017. ISBN: 9781292158587 (cit. on p. 3).

[31] J. Pérez, M. Arenas, and C. Gutierrez. "Semantics and Complexity of SPARQL". In: *Semantics and complexity of SPARQL. ACM Trans. Database Syst* 34.16 (2009). DOI: 10.1145/1567274.1567278 (cit. on p. 18).

[32] F. Picalausa and S. Vansummeren. "What are real SPARQL queries like?" In: *Proceedings of the International Workshop on Semantic Web Information Management*. SWIM '11 Article 7. Athens, Greece: Association for Computing Machinery, June 2011, pp. 1–6. ISBN: 9781450306515. DOI: 10.1145/1999299.1999306 (cit. on pp. 16, 18).

[33] Security Compass. *The 2021 State of Cloud Adoption*. en. https://resources.securitycompass.com/reports/2021-state-of-cloud-adoption. Accessed: 2022-2-9. Sept. 2021 (cit. on p. 2).

[34] N. Sioutos et al. "NCI Thesaurus: a semantic model integrating cancer-related clinical and molecular information". en. In: *J. Biomed. Inform.* 40.1 (Feb. 2007), pp. 30–43. ISSN: 1532-0464, 1532-0480. DOI: 10.1016/j.jbi.2006.02.013 (cit. on p. 1).

[35] D. X. Song, D. Wagner, and A. Perrig. "Practical techniques for searches on encrypted data". In: *Proceeding 2000 IEEE Symposium on Security and Privacy. S P 2000*. May 2000, pp. 44–55. DOI: 10.1109/SECPRI.2000.848445 (cit. on p. 3).

[36] R. Studer, V. R. Benjamins, and D. Fensel. "Knowledge engineering: Principles and methods". In: *Data Knowl. Eng.* 25.1 (Mar. 1998), pp. 161–197. ISSN: 0169-023X. DOI: 10.1016/S0169-023X(97)00056-6 (cit. on p. 10).

[37] The W3C SPARQL Working Group. *SPARQL 1.1 Overview*. https://www.w3.org/TR/sparql11-overview/. Mar. 2013 (cit. on pp. 3, 15).

[38] T. Tudorache, J. Vendetti, and N. F. Noy. "Web-Protégé: A lightweight OWL ontology editor for the web". In: *CEUR Workshop Proceedings*. Vol. 432. 2009 (cit. on p. 2).

[39] T. Tudorache et al. "Supporting collaborative ontology development in Protégé". In: *Lect. Notes Comput. Sci.* 5318 LNCS (2008), pp. 17–32. ISSN: 0302-9743, 1611-3349. DOI: 10.1007/978-3-540-88564-1_2 (cit. on p. 2).

[40] W3C. *Extensible markup language (XML) 1.0 (fifth edition)*. en. https://www.w3.org/TR/xml/. Accessed: 2022-2-11 (cit. on p. 7).

[41] W3C. *OWL 2 web ontology language direct semantics (second edition)*. en. https://www.w3.org/TR/2012/REC-owl2-direct-semantics-20121211/. Accessed: 2022-2-14 (cit. on pp. 10, 11).

[42] W3C. *OWL 2 web ontology language document overview (second edition)*. en. https://www.w3.org/TR/owl2-overview/. Accessed: 2022-2-11 (cit. on pp. 7, 10, 11).

[43] W3C. *OWL 2 web ontology language Manchester syntax (second edition)*. en. https://www.w3.org/TR/2012/NOTE-owl2-manchester-syntax-20121211/. Accessed: 2022-2-14 (cit. on p. 11).

[44] W3C. *OWL 2 web ontology language profiles (second edition)*. en. https://www.w3.org/TR/2012/REC-owl2-profiles-20121211/. Accessed: 2022-2-14 (cit. on p. 11).

[45] W3C. *OWL 2 web ontology language quick reference guide (second edition)*. en. https://www.w3.org/TR/owl2-quick-reference/. Accessed: 2022-2-14 (cit. on p. 11).

[46] W3C. *OWL 2 web ontology language RDF-based semantics (second edition)*. en. https://www.w3.org/TR/2012/REC-owl2-rdf-based-semantics-20121211/. Accessed: 2022-2-14 (cit. on p. 10).

[47] W3C. *OWL 2 web ontology language structural specification and functional-style syntax (second edition)*. en. https://www.w3.org/TR/2012/REC-owl2-syntax-20121211/. Accessed: 2022-2-14 (cit. on p. 11).

[48] W3C. *OWL 2 web ontology language XML serialization (second edition)*. en. https://www.w3.org/TR/2012/REC-owl2-xml-serialization-20121211/. Accessed: 2022-2-14 (cit. on p. 11).

[49] W3C. *RDF 1.1 concepts and abstract syntax*. en. https://www.w3.org/TR/rdf11-concepts/. Accessed: 2022-2-11 (cit. on pp. 7, 8).

[50] W3C. *RDF 1.1 TriG*. en. https://www.w3.org/TR/trig/. Accessed: 2022-2-13 (cit. on p. 8).

[51] W3C. *RDF 1.1 turtle*. en. https://www.w3.org/TR/turtle/. Accessed: 2022-2-13 (cit. on p. 8).

[52] W3C. *RDF 1.1 XML syntax*. en. https://www.w3.org/TR/rdf-syntax-grammar/. Accessed: 2022-2-13 (cit. on p. 9).

[53] W3C. *RDF Schema 1.1*. en. https://www.w3.org/TR/rdf-schema/. Accessed: 2022-2-11 (cit. on pp. 7, 9).

[54] W3C. *RDFa core 1.1 - third edition*. en. https://www.w3.org/TR/rdfa-core/. Accessed: 2022-2-13 (cit. on p. 9).

[55] W3C. *Semantic Web Case Studies and Use Cases*. en. https://www.w3.org/2001/sw/sweo/public/UseCases/. Accessed: 2022-2-8 (cit. on p. 1).

[56] W3C. *SPARQL 1.1 update*. en. https://www.w3.org/TR/sparql11-update/. Accessed: 2022-2-15 (cit. on p. 15).

[57] W3C. *SPARQL Query Language for RDF*. https://www.w3.org/TR/rdf-sparql-query/. Jan. 2008 (cit. on pp. 15, 16).

[58] W3C. *SWRL: A Semantic Web Rule Language Combining OWL and RuleML*. https://www.w3.org/Submission/SWRL/. May 2004 (cit. on p. 7).

[59] Wikipedia contributors. *Semantic web stack*. https://en.wikipedia.org/wiki/File:Semantic_web_stack.svg. Accessed: 2022-2-15 (cit. on p. 7).